



1. 串的定义和基本操作

串 是零个或多个字符组成的有限序列

$$s = "a_1 a_2 a_3 \dots a_n" \quad (n > 0)$$

s: 串的名

"a₁a₂a₃...a_n": 串的值

a_i: 字符 (字母、数字.....)

n: 串的长度



1. 串的定义和基本操作

串的基本操作

StrAssign (&T,chars) : 赋值操作。将串T赋值为chars。

StrCopy (&T,S) : 复制操作。将串S复制给串T。

StrEmpty (S) : 判空操作。若S为空串，则返回TRUE，否则返回FALSE。

strcmpare (S,T) : 比较操作。若 $S>T$ ，返回值 >0 ；若 $S=T$ ，返回值 $=0$ ；若 $S<T$ ，返回值 <0 。

StrLength (S) : 求串长操作。返回串S中字符的个数。

substring (&Sub,S,pos,len) : 求子串操作。用Sub返回串S中从第pos个字符起、长度为len的子串。

Concat (&T,S1,S2) : 串联接操作。用T返回由S1 S2联接而成的新串。

Index (S,T) : 定位操作。若主串S中存在与T值相同的子串，则返回其在S中首次出现的位置；否则返回0。

ClearString (&S) : 清空操作。将串S置为空串。

Destroyststring (&S) : 销毁操作。将串S销毁。



2. 串的存储结构

① 串的顺序存储

```
//串的顺序存储
#define MAXLEN 255 //串的最大长度
typedef struct{
    char ch[MAXLEN + 1]; //存储串的一维数组
    int length; //串的当前长度
}SString;
```

```
//串的堆式顺序存储
typedef struct{
    char *ch; //若是非空串，则按串长度分配存储区，否则ch为NULL
    int length; //串的当前长度
}HString;
```



2. 串的存储结构

②串的链式存储

```
#define CHUNKSIZE 80 //可由用户定义的块的大小
typedef struct Chunk{
    char ch[CHUNKSIZE];
    struct Chunk *next;
}Chunk;
typedef struct{
    Chunk *head, *tail; //串的头和尾指针
    int length; //串的当前长度
}
```



4. 串的模式匹配算法-KMP

① KMP的基本思想

KMP

利用已匹配部分的信息，在失配时尽可能多地保留已有成果，避免主串回退和重复比较



4. 串的模式匹配算法-KMP

① KMP的基本思想



失配以后，不是从头乱试，而是把模式串前面那段还能用的前缀，直接挪到主串后面那段已经匹配好的后缀上
寻找一个最长的公共前后缀，并把前缀对齐到已匹配部分的后缀处

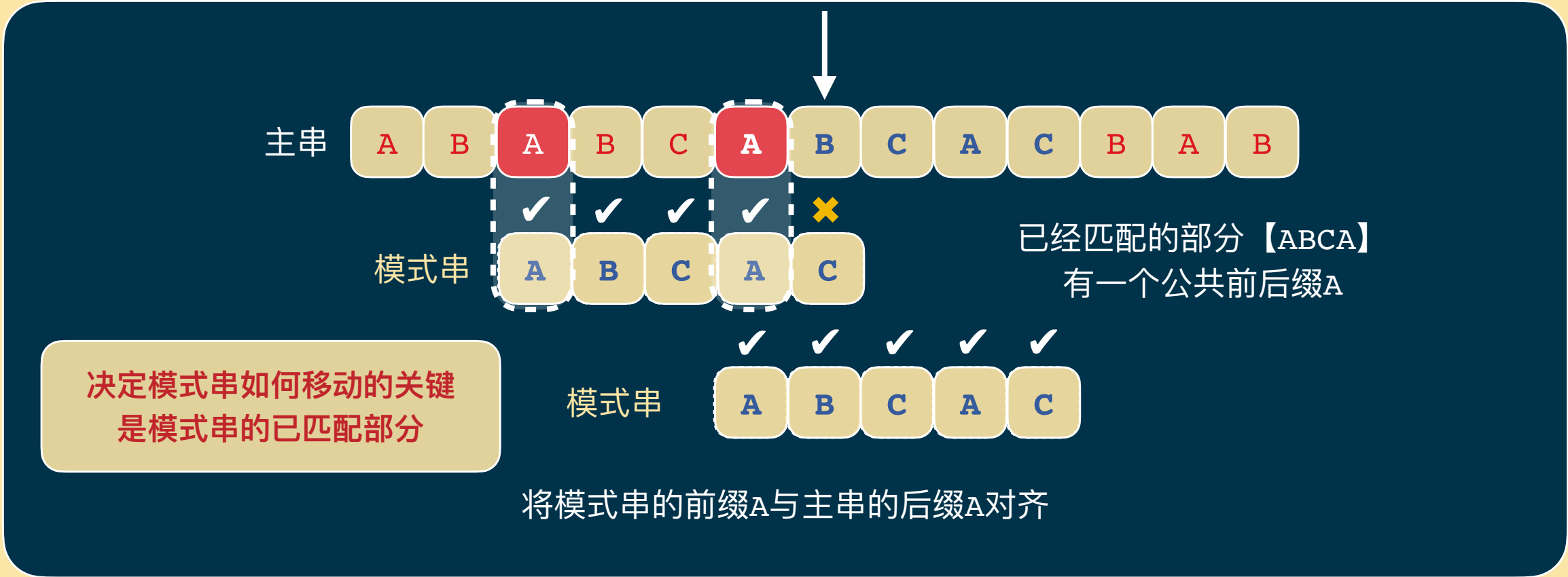
KMP 利用已匹配部分的信息，在失配时尽可能多地保留已有成果，避免主串回退和重复比较



4. 串的模式匹配算法-KMP

②next数组

失配以后，不是从头乱试，而是把模式串前面那段还能用的前缀，直接挪到主串后面那段已经匹配好的后缀上
寻找一个最长的公共前后缀，并把前缀对齐到已匹配部分的后缀处





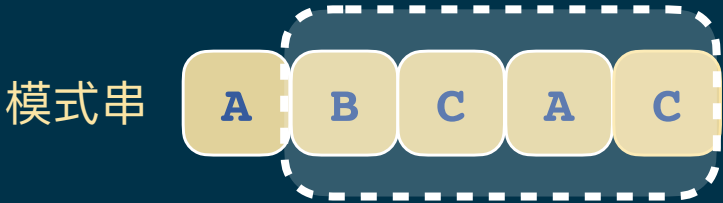
4. 串的模式匹配算法-KMP

②next数组

失配以后，不是从头乱试，而是把模式串前面那段还能用的前缀，直接挪到主串后面那段已经匹配好的后缀上
寻找一个最长的公共前后缀，并把前缀对齐到已匹配部分的后缀处



将前缀A移动到原本后缀A的位置



主串指针之前的部分一定匹配
因此从当前位置开始比对即可

决定模式串如何移动的关键
是模式串的已匹配部分



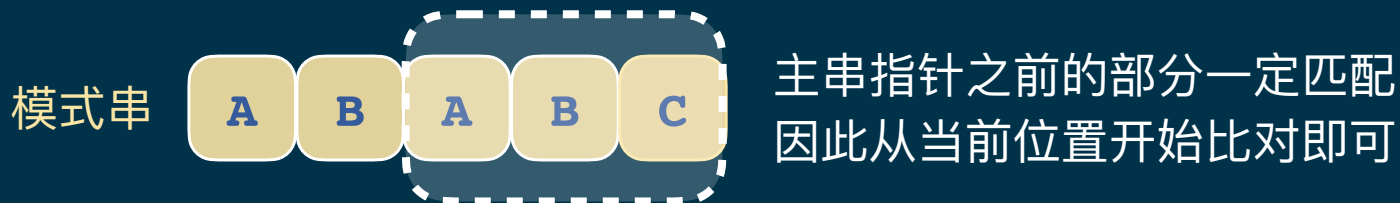
4. 串的模式匹配算法-KMP

②next数组

失配以后，不是从头乱试，而是把模式串前面那段还能用的前缀，直接挪到主串后面那段已经匹配好的后缀上
寻找一个最长的公共前后缀，并把前缀对齐到已匹配部分的后缀处



将前缀AB移动到原本后缀AB的位置

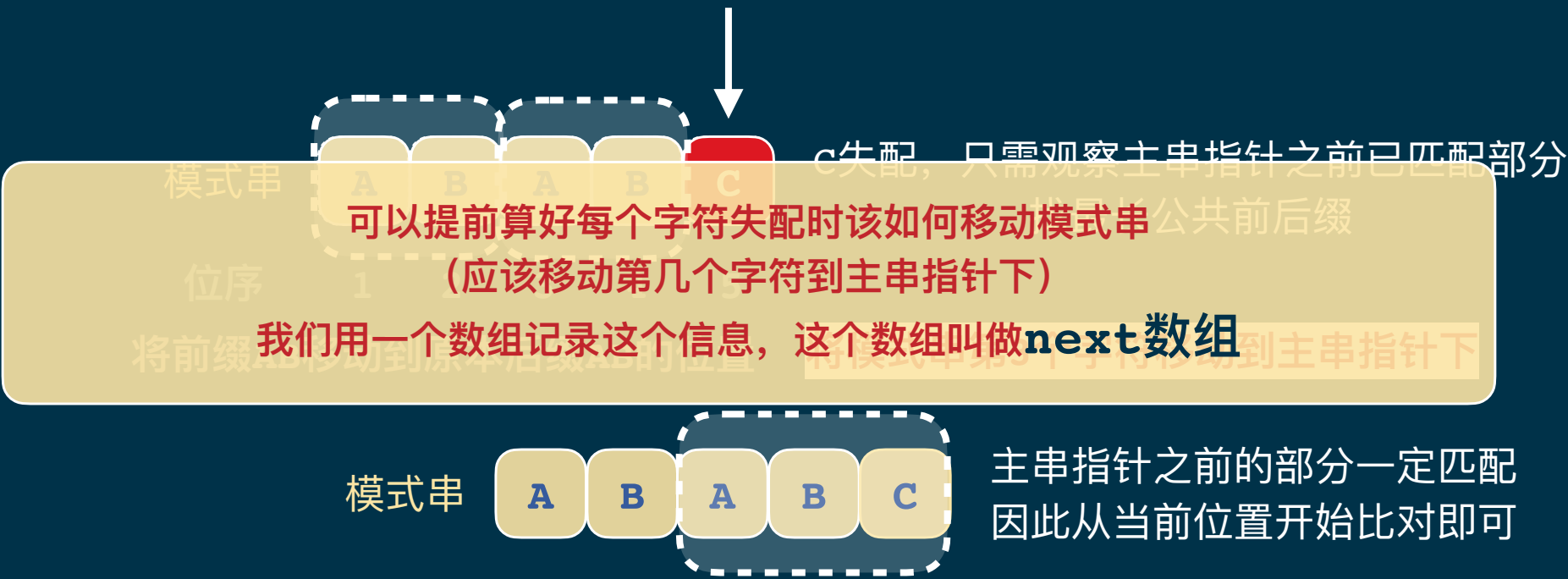




4. 串的模式匹配算法-KMP

②next数组

失配以后，不是从头乱试，而是把模式串前面那段还能用的前缀，直接挪到主串后面那段已经匹配好的后缀上
寻找一个最长的公共前后缀，并把前缀对齐到已匹配部分的后缀处





4. 串的模式匹配算法-KMP

②next数组

可以提前算好每个字符失配时该如何移动模式串
(应该移动第几个字符到主串指针下)

我们用一个数组记录这个信息，这个数组叫做**next**数组

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1			

第1个字符对应的next数组值一定是0

第2个字符对应的next数组值一定是1



4. 串的模式匹配算法-KMP

②next数组

可以提前算好每个字符失配时该如何移动模式串
(应该移动第几个字符到主串指针下)

我们用一个数组记录这个信息，这个数组叫做**next**数组

tips:

- 1. 自己不能是自己的前缀和后缀
- 2. 前缀和后缀的阅读方向一致

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1	1	1	2

从第3个字符开始
next数组值为【前面子串的最大公共前后缀长度+1】



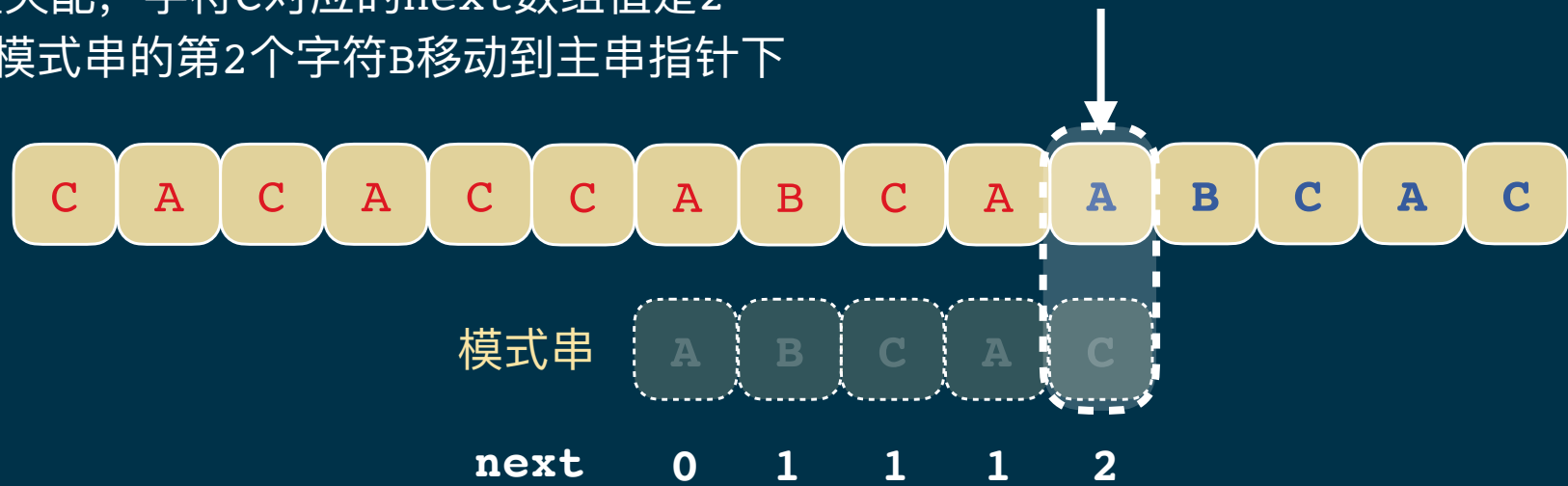
4. 串的模式匹配算法-KMP

②next数组

可以提前算好每个字符失配时该如何移动模式串
(应该移动第几个字符到主串指针下)

我们用一个数组记录这个信息，这个数组叫做next数组

当前位置失配，字符C对应的next数组值是2
因此应该将模式串的第2个字符B移动到主串指针下



观察一下
其实就是把前缀A对齐了后缀A
完全没问题，说明next数组的作用达成！





4. 串的模式匹配算法-KMP

②next数组

可以提前算好每个字符失配时该如何移动模式串
(应该移动第几个字符到主串指针下)

我们用一个数组记录这个信息，这个数组叫做**next**数组

下标	0	1	2	3	4
模式串数组	A	B	C	A	C
next	0	1	1	1	2
	-1	0	0	0	1

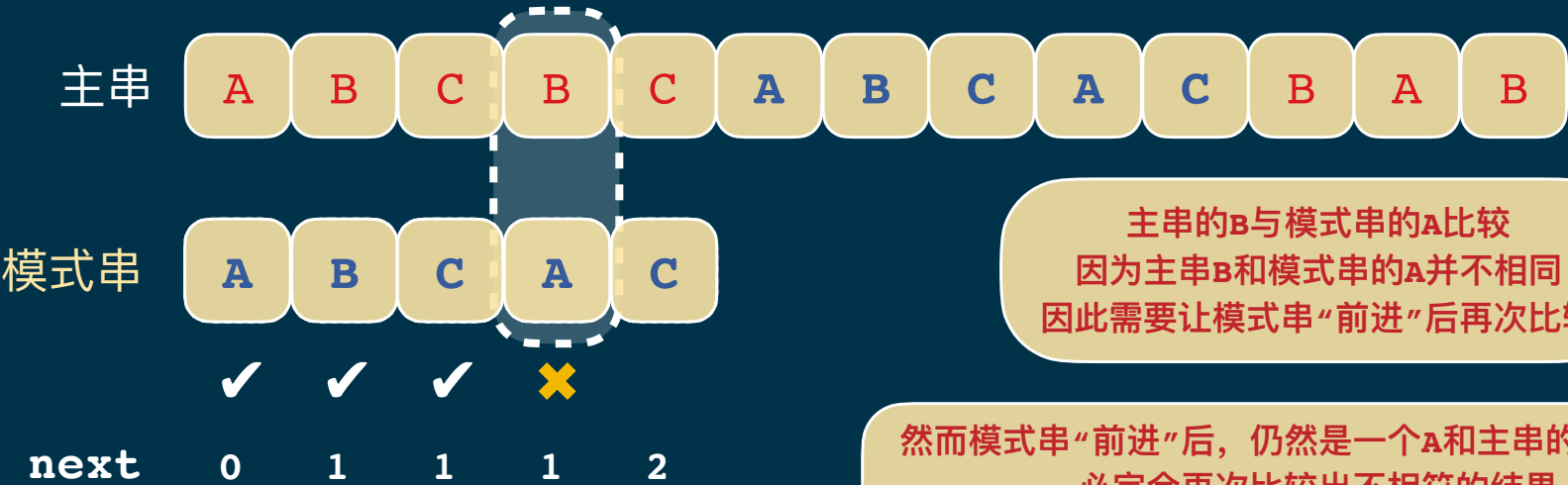
next数组值告诉我们应该移动模式串的第几个字符到指针下

next数组值告诉我们应该移动模式串下标为几的字符到指针下



4. 串的模式匹配算法-KMP

③nextval数组



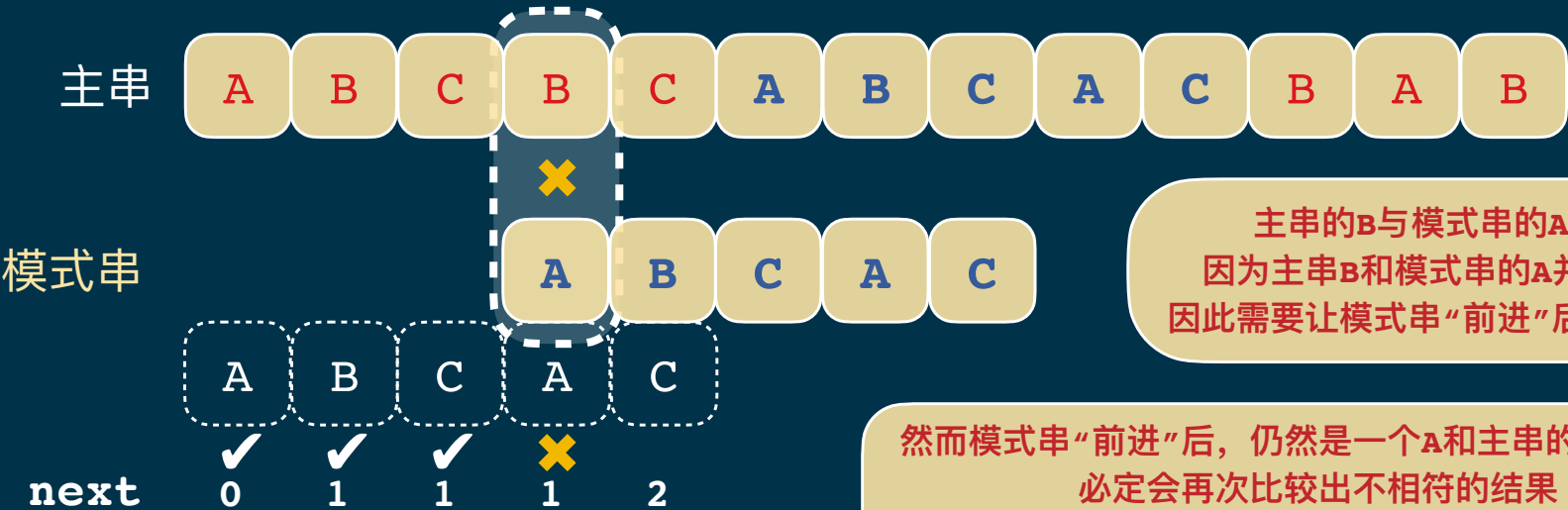
主串的B与模式串的A比较
因为主串B和模式串的A并不相同
因此需要让模式串“前进”后再次比较

然而模式串“前进”后，仍然是一个A和主串的B进行比较
必定会再次比较出不相符的结果
所以这一次比较是没有意义的



4. 串的模式匹配算法-KMP

③nextval数组



主串的B与模式串的A比较
因为主串B和模式串的A并不相同
因此需要让模式串“前进”后再次比较

然而模式串“前进”后，仍然是一个A和主串的B进行比较
必定会再次比较出不相符的结果
所以这一次比较是没有意义的



4. 串的模式匹配算法-KMP

③nextval数组

nextval数组的求法：

依次观察模式串的每一位：

对比【本位字符】和【本位字符的next数组值对应的字符】。

如果二者相同，则nextval值改为next数组值对应的字符的nextval数组值。

如果二者不同，nextval值和本位的next数组值相同。





4. 串的模式匹配算法-KMP

③nextval数组

nextval数组的求法：

依次观察next数组每一位。

对比【本位字符】和【本位字符的next数组值对应的字符】。

如果二者相同，则nextval值改为next数组值对应的字符的nextval数组值。

如果二者不同，nextval值和本位的next数组值相同。





4. 串的模式匹配算法-KMP

③nextval数组

nextval数组的求法：

依次观察next数组每一位。

对比【本位字符】和【本位字符的next数组值对应的字符】。

如果二者相同，则nextval值改为next数组值对应的字符的nextval数组值。

如果二者不同，nextval值和本位的next数组值相同。





4. 串的模式匹配算法-KMP

③nextval数组

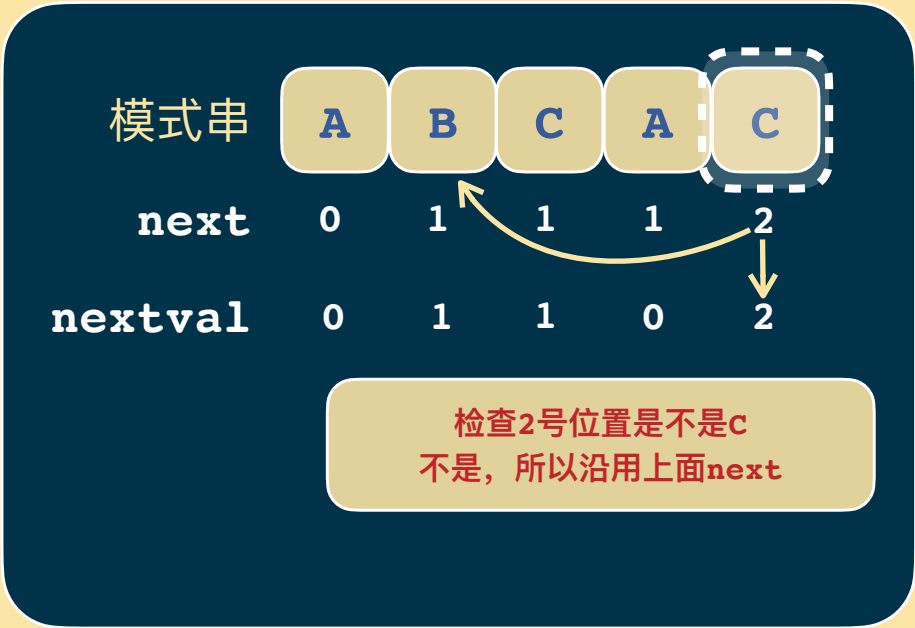
nextval数组的求法：

依次观察next数组每一位。

对比【本位字符】和【本位字符的next数组值对应的字符】。

如果二者相同，则nextval值改为next数组值对应的字符的nextval数组值。

如果二者不同，nextval值和本位的next数组值相同。





真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 设主串 $T = \text{"abaabaabcabaabc"}$, 模式串 $S = \text{"abaabc"}$, 采用 KMP 算法进行模式匹配, 到匹配成功时为止, 在匹配过程中进行的单个字符间的比较次数是

A. 9

B. 10

C. 12

D. 15



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2.KMP算法使用修正后的next数组进行模式匹配，模式串S="aabaab"，当主串中某字符与s中某字符失配时，s将向右滑动的最长距离是

A.5

B.4

C.3

D.2